

Data Bricks E-commerce Analytics Project Report

Prabudh Raghuram

Platform: Databricks Lakehouse Platform

Technologies: SQL, Python, PySpark, Pandas, Delta Lake, Scikit-learn, Matplotlib

1. Executive Summary

This report documents the complete lifecycle of a data analytics project aimed at deriving actionable insights from a large, real-world e-commerce dataset. Executed entirely within the Databricks Lakehouse Platform, the project transforms over one million raw transactional records into a valuable business asset. The primary outcomes include a robust data pipeline following the Medallion Architecture, a detailed customer segmentation using RFM analysis, and a predictive machine learning model to forecast customer spending. This project successfully demonstrates the power of unified analytics in solving complex data challenges, moving from raw data ingestion and cleaning to advanced modelling and visualization.

2. Introduction & Project Objectives

The modern e-commerce landscape is fiercely competitive, making data-driven decision-making a critical factor for success. The objective of this project was to analyse a UK-based online retailer's transactional data to:

Engineer a Reliable Data Pipeline: Build a scalable, multi-stage pipeline to handle complex raw data, address quality issues, and prepare it for analysis.

Segment Customers: Use RFM (Recency, Frequency, Monetary) analysis to group customers into meaningful segments (e.g., "Champions," "At Risk") that can inform targeted marketing strategies.

Predict Customer Value: Develop a machine learning model to predict the potential monetary value of a customer based on their historical behaviour.

3. Methodology: The Medallion Architecture

To ensure data quality, reliability, and governance, this project was structured around the Medallion Architecture. This industry best practice organizes data into three distinct layers, progressively improving its quality and usability.

 **Bronze Layer:** Contains raw, unaltered data ingested directly from source systems. This layer serves as the historical archive and the single source of truth, allowing for data to be reprocessed if business logic changes

2 Silver Layer: Data from the Bronze layer is cleaned, validated, filtered, and enriched. This layer represents a clean, conformed, and trustworthy version of the data, ready for analysis across the business.

1 Gold Layer: Data from the Silver layer is aggregated and transformed to serve specific business use cases. These tables are often organized by business function and are used for reporting, analytics, and powering machine learning models.

4. Project Execution & Workflow

Phase 1: Ingestion (Bronze Layer)

The project began with the most critical and challenging phase: ingesting the source data ,a 43.51 MB multi-sheet Excel file.

Challenges & Solutions:

Data Truncation: Initial attempts to convert the .xlsx file's sheets to .csv resulted in silent data truncation, with the export process failing to include the final ~40,000 rows of data.

File Type Incompatibility: The standard Databricks UI for creating tables does not support .xlsx files.

Solution: The robust solution was to upload the original .xlsx file to a Databricks Volume and read the data from each sheet directly using a Python script with the pandas and PySpark libraries. This bypassed the faulty CSV conversion entirely.

Mixed Data Types: During the read process, PySparkTypeError exceptions occurred due to columns containing both text and numbers (e.g., Invoice containing 536365 and C536365). This was solved by explicitly defining the data type for these columns as string upon-ingestion.

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer
1	537131	22121	NOEL WOODEN BLOCK LETTERS	2	2010-12-05T12:26:00.000+00:00	5.99	15716
2	537131	21326	AGED GLASS SILVER T-LIGHT HOLDER	12	2010-12-05T12:26:00.000+00:00	0.65	15716
3	537131	22809	SET OF 6 T-LIGHTS SANTA	6	2010-12-05T12:26:00.000+00:00	2.95	15716
4	537131	22572	ROCKING HORSE GREEN CHRISTMAS	96	2010-12-05T12:26:00.000+00:00	0.72	15716
5	537131	22576	SWALLOW WOODEN CHRISTMAS DECORATION	48	2010-12-05T12:26:00.000+00:00	0.85	15716
6	537131	22086	PAPER CHAIN KIT 50'S CHRISTMAS	2	2010-12-05T12:26:00.000+00:00	2.95	15716
7	537131	22910	PAPER CHAIN KIT VINTAGE CHRISTMAS	1	2010-12-05T12:26:00.000+00:00	2.95	15716
8	537131	22365	DOORMAT RESPECTABLE HOUSE	1	2010-12-05T12:26:00.000+00:00	7.95	15716
9	537131	22904	CALENDAR PAPER CUT DESIGN	3	2010-12-05T12:26:00.000+00:00	2.95	15716
10	537131	22121	NOEL WOODEN BLOCK LETTERS	1	2010-12-05T12:26:00.000+00:00	5.99	15716
11	537131	22470	HEART OF WICKER LARGE	5	2010-12-05T12:26:00.000+00:00	2.95	15716
12	537131	22739	RIBBON RFFI CHRISTMAS SOCK BAUBLE	10	2010-12-05T12:26:00.000+00:00	1.65	15716
13	537131	22686	WICKER WREATH LARGE	4	2010-12-05T12:26:00.000+00:00	1.95	15716
14	537131	22155	STAR DECORATION RUSTIC	2	2010-12-05T12:26:00.000+00:00	0.42	15716
15	537131	22604	SET OF 4 NAPKIN CHARMS CUTLERY	1	2010-12-05T12:26:00.000+00:00	2.55	15716
16	537131	22638	SET OF 4 NAPKIN CHARMS CROWNS	1	2010-12-05T12:26:00.000+00:00	2.55	15716
17	537131	22639	SET OF 4 NAPKIN CHARMS HEARTS	3	2010-12-05T12:26:00.000+00:00	2.55	15716
18	537131	22585	PACK OF 6 BIRDY GIFT TAGS	6	2010-12-05T12:26:00.000+00:00	1.25	15716
19	537131	22574	HEART WOODEN CHRISTMAS DECORATION	96	2010-12-05T12:26:00.000+00:00	0.80	15716
20	537131	21294	ETCHED GLASS COASTER	48	2010-12-05T12:26:00.000+00:00	1.25	15716
21	537131	85049A	TRADITIONAL CHRISTMAS RIBBONS	6	2010-12-05T12:26:00.000+00:00	1.25	15716
22	537131	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	3	2010-12-05T12:26:00.000+00:00	3.75	15716
23	537131	22885	HOT WATER BOTTLE I AM SO POORLY	2	2010-12-05T12:26:00.000+00:00	4.65	15716

Outcome: A complete, correct bronze_online_retail Delta table was created, containing all 1,067,371 raw records.

Phase 2: Cleansing & Enrichment (Silver Layer)

The raw bronze data was transformed into a clean, analysis-ready Silver table using a single SQL query. This involved:

Filtering: Removing records with NULL CustomerIDs and transactions that were returns (where Quantity < 0).

Enrichment: Creating a new TotalPrice column by multiplying Quantity by Price.

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer
1	489434	85048	TSCM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01T07:45:00.000+00:00	6.95	13085
2	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01T07:45:00.000+00:00	6.75	13085
3	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01T07:45:00.000+00:00	6.75	13085
4	489434	22041	RECORD FRAME 17 SINGLE SIZE	48	2009-12-01T07:45:00.000+00:00	2.1	13085
5	489434	21732	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01T07:45:00.000+00:00	1.25	13085
6	489434	22064	PINK DOUGHNUT TRINKET POT	24	2009-12-01T07:45:00.000+00:00	1.65	13085
7	489434	21871	SAVE THE PLANET MUG	24	2009-12-01T07:45:00.000+00:00	1.25	13085
8	489434	21523	FANCY FONT HOME SWEET HOME DOORMAT	10	2009-12-01T07:45:00.000+00:00	5.95	13085
9	489435	22350	CAT BOWL	12	2009-12-01T07:46:00.000+00:00	2.55	13085
10	489435	22349	DOG BOWL, CHASING BALL DESIGN	12	2009-12-01T07:46:00.000+00:00	3.75	13085
11	489435	22195	HEART MEASURING SPOONS LARGE	24	2009-12-01T07:46:00.000+00:00	1.65	13085
12	489435	22353	LUNCHBOX WITH CUTLERY FAIRY CAKES	12	2009-12-01T07:46:00.000+00:00	2.55	13085
13	489436	48173C	DOOR MAT BLACK FLOCK	10	2009-12-01T09:06:00.000+00:00	5.95	13078
14	489436	21755	LOVE BUILDING BLOCK WORD	18	2009-12-01T09:06:00.000+00:00	5.45	13078
15	489436	21754	HOME BUILDING BLOCK WORD	3	2009-12-01T09:06:00.000+00:00	5.95	13078
16	489436	84879	ASSORTED COLOUR BIRD ORNAMENT	16	2009-12-01T09:06:00.000+00:00	1.68	13078
17	489436	22119	PEACE WOODEN BLOCK LETTERS	3	2009-12-01T09:06:00.000+00:00	6.95	13078
18	489436	22142	CHRISTMAS CRAFT WHITE FAIRY	12	2009-12-01T09:06:00.000+00:00	1.45	13078
19	489436	22296	HEART IVORY TRELLIS LARGE	12	2009-12-01T09:06:00.000+00:00	1.65	13078
20	489436	22295	HEART FUGREE DOVE LARGE	12	2009-12-01T09:06:00.000+00:00	1.65	13078
21	489436	22109	FULL ENGLISH BREAKFAST PLATE	16	2009-12-01T09:06:00.000+00:00	3.39	13078
22	489436	22107	PIZZA PLATE IN BOX	4	2009-12-01T09:06:00.000+00:00	3.75	13078
23	489436	22194	BLACK DINER WALL CLOCK	2	2009-12-01T09:06:00.000+00:00	8.5	13078
24	489436	22045	PEACE WOODEN BLOCK LETTERS	3	2009-12-01T09:06:00.000+00:00	6.95	13078

Outcome: The silver_online_retail table was created, containing 824,364 valid, analysable transactions.

Phase 3: Aggregation & Segmentation (Gold Layer)

The clean silver data was aggregated to a customer-centric view using RFM analysis.

RFM Calculation: A SQL query grouped the data by CustomerID to calculate their Recency, Frequency, and Monetary values.

Scoring: The NTILE(4) window function was used to assign a score from 1-4 for each RFM metric.

Segmentation: A CASE statement applied business logic to the scores to assign human-readable segments like "Champions," "Loyal Customers," and "At Risk."

Outcome: A final gold_customer_segments table that provides actionable insights for marketing and sales teams.

dbc-5a0194dd-2cae.cloud.databricks.com/editor/queries/21142275628369767o=3115700028360582

Search data, notebooks, recent, and more... CTRL + P

workspace

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

AI/ML

Playground

Experiments

Features

Models

Serving

Run all (1000) 11:30 AM (UTC) workspace default New SQL editor: ON Last edit was 15 minutes ago

```

71 -- Or use logic to create named segments
72 CASE
73 WHEN R_Score = 4 AND F_Score = 4 AND M_Score = 4 THEN 'Champions'
74 WHEN R_Score >= 3 AND F_Score >= 3 THEN 'Loyal Customers'
75 WHEN R_Score >= 3 AND F_Score <= 2 THEN 'Potential Loyalists'
76 WHEN R_Score = 4 AND F_Score = 1 THEN 'New Customers'
77 WHEN R_Score = 2 AND (F_Score = 1 OR F_Score = 2) THEN 'Promising'
78 WHEN R_Score = 3 AND (F_Score = 1 OR F_Score = 2) THEN 'Needs Attention'
79 WHEN R_Score <= 2 AND F_Score >= 3 THEN 'At Risk (High Value)'
80 WHEN R_Score <= 2 AND F_Score <= 2 THEN 'At Risk (Low Value)'
81 WHEN R_Score = 1 THEN 'Lost'

```

Add parameter

	CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Score_Combined	Customer_Segment
1	18102	0	145	608821.6999999999	4	4	4	444	Champions
2	14646	1	152	528602.5999999998	4	4	4	444	Champions
3	14156	9	156	313946.3699999999	4	4	4	444	Champions
4	14911	1	398	295972.6300000000	4	4	4	444	Champions
5	17450	8	51	246873.0699999999	4	4	4	444	Champions
6	13604	3	143	196482.8100000001	4	4	4	444	Champions
7	17511	2	60	175603.5499999999	4	4	4	444	Champions
8	16446	0	2	168472.5	4	2	4	424	Potential Loyalists
9	16684	4	55	147142.7700000000	4	4	4	444	Champions
10	12415	24	28	144458.3699999999	4	4	4	444	Champions
11	15061	3	127	137818.5199999999	4	4	4	444	Champions
12	16029	38	107	122208.1400000000	3	4	4	344	Loyal Customers
13	17949	1	118	118628.0799999999	4	4	4	444	Champions
14	15311	0	208	116771.1600000000	4	4	4	444	Champions
15	13089	2	203	116757.8600000000	4	4	4	444	Champions

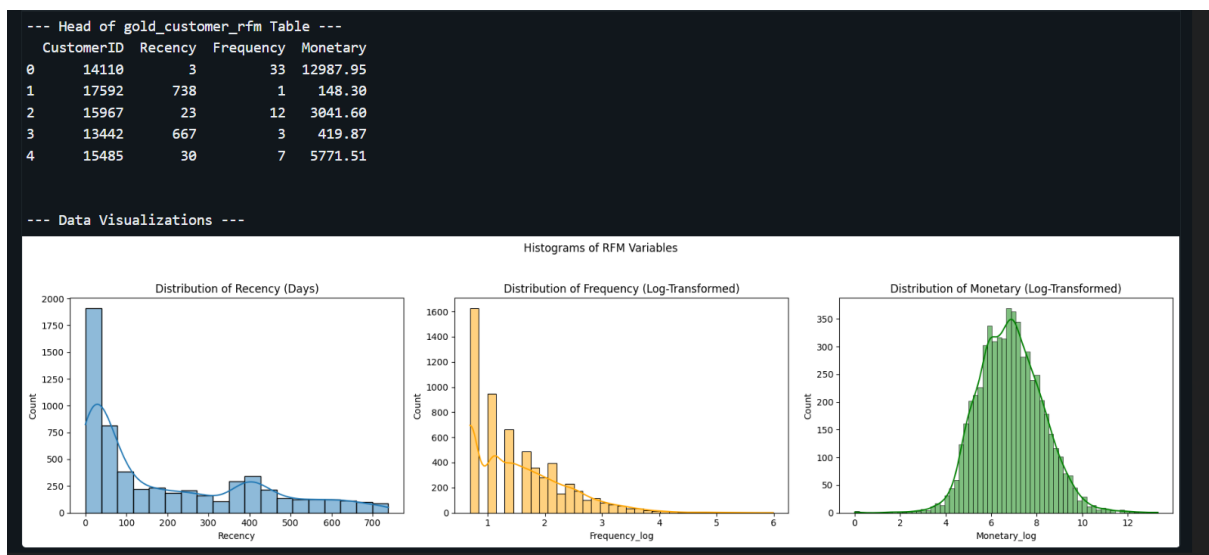
1,000 rows | 0.91s runtime

Refreshed 15 minutes ago

Phase 4: Visualization & Predictive Modelling

The final phase focused on data science using Python.

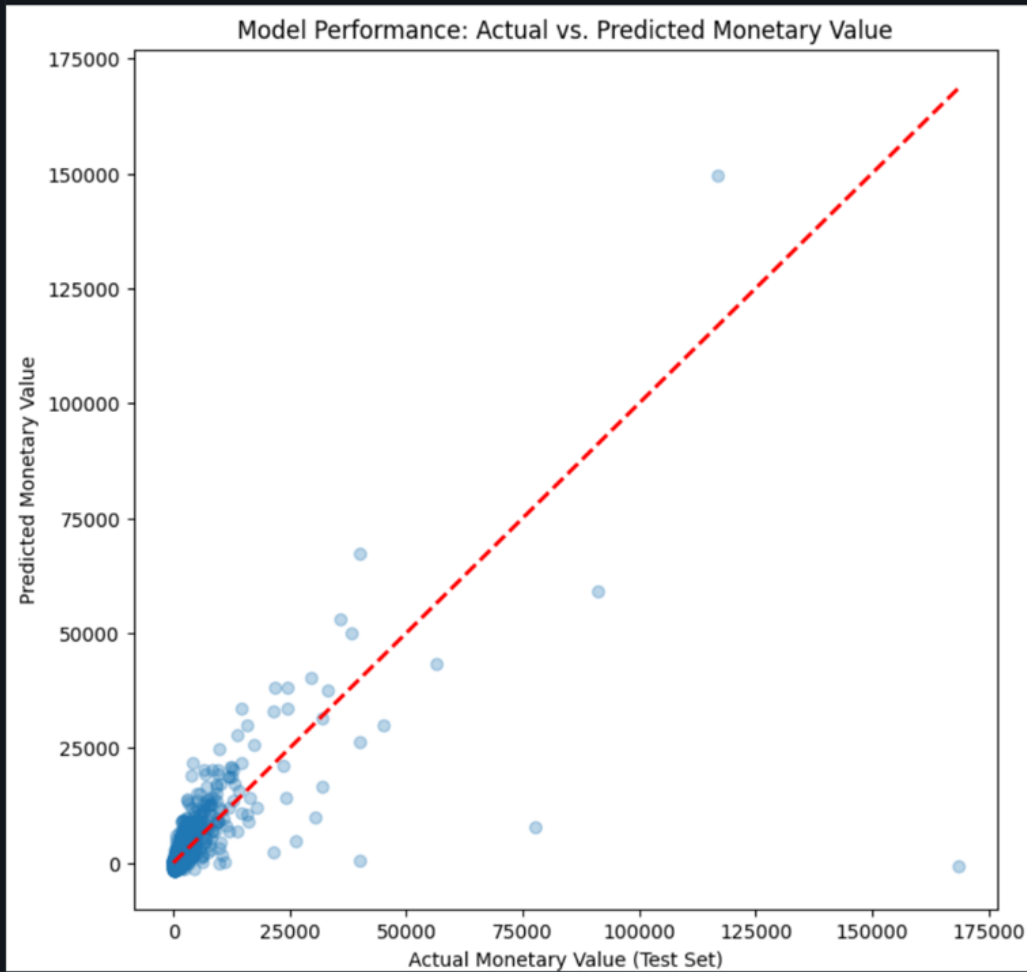
Data Visualization: Histograms were generated to understand the distribution of the RFM variables. This revealed that Frequency and Monetary were heavily skewed, a common pattern in transactional data.



Predictive Modelling: A Linear Regression model was trained using scikit-learn to predict a customer's Monetary value based on their Recency and Frequency.

Model Performance: The model's accuracy was evaluated by plotting its predicted values against the actual values from a held-out test set. The plot shows a strong positive correlation, indicating that the model successfully learned the general relationship between customer behaviour and spending.

--- Machine Learning Model Trained Successfully ---
Model formula: Monetary = (Recency * 3.00) + (Frequency * 748.53) + -2279.29



5. Conclusion

This project successfully demonstrates a complete, end-to-end data workflow within the Databricks Lakehouse Platform. By systematically progressing through the Bronze, Silver, and Gold layers of the Medallion Architecture, we transformed a complex, error-prone dataset into a reliable source for advanced analytics. The final outputs a detailed customer segmentation and a predictive spending model provide valuable, actionable intelligence that can directly inform business strategy, optimize marketing campaigns, and drive growth in an e-commerce environment. The challenges overcome, particularly during the ingestion phase, highlight the importance of robust data engineering practices as the foundation for any successful data science initiative.

6. Appendix: Complete Project Code

A. Code for Phase 1: Ingestion (Python)

```
# --- Configuration ---
```

```
excel_file_path = "/Volumes/workspace/default/retail_data/online_retail_II.xlsx"
```

```
bronze_table_name = "bronze_online_retail"
```

```
# -----
```

```
# 1. Install library for reading Excel files
```

```
%pip install openpyxl
```

```
import pandas as pd
```

```
# 2. Define explicit data types for columns known to have mixed types.
```

```
col_types = {
```

```
    'Invoice': str,
```

```
    'StockCode': str,
```

```
    'Customer ID': str,
```

```
    'Description': str
```

```
}
```

```
# 3. Read the 'Year 2009-2010' sheet.
```

```
print("Reading first sheet ('Year 2009-2010')...")
```

```
pandas_df1 = pd.read_excel(excel_file_path, sheet_name='Year 2009-2010',  
dtype=col_types)
```

```
spark_df1 = spark.createDataFrame(pandas_df1)
```

```
# 4. Read the 'Year 2010-2011' sheet.
```

```
print("Reading second sheet ('Year 2010-2011')...")
```

```

pandas_df2 = pd.read_excel(excel_file_path, sheet_name='Year 2010-2011',
dtype=col_types)

spark_df2 = spark.createDataFrame(pandas_df2)

# 5. Combine the two DataFrames.

print("Combining data from both sheets...")

raw_df = spark_df1.unionByName(spark_df2)

# 6. Rename the 'Customer ID' column.

raw_df_renamed = raw_df.withColumnRenamed("Customer ID", "CustomerID")

# 7. Write the final DataFrame to a Bronze Delta table.

print(f"Saving data to Bronze table: '{bronze_table_name}'...")

raw_df_renamed.write.format("delta").mode("overwrite").saveAsTable(bronze_table_name)

print(f"\n--- SUCCESS! ---")

print(f"Bronze table '{bronze_table_name}' has been created with
{raw_df_renamed.count()} total rows.")

```

B. Code for Phases 2 & 3: Silver, Gold & Segmentation (SQL)

-- Step 1: Create the Silver Layer

```

CREATE OR REPLACE TABLE silver_online_retail AS

SELECT

*,

Quantity * Price AS TotalPrice

FROM

```

bronze_online_retail

WHERE

CustomerID IS NOT NULL

AND Quantity > 0;

-- Step 2: Create the Gold RFM Layer

CREATE OR REPLACE TABLE gold_customer_rfm AS

SELECT

CustomerID,

DATEDIFF(

(

SELECT

MAX(InvoiceDate)

FROM

silver_online_retail

),

MAX(InvoiceDate)

) AS Recency,

COUNT(DISTINCT Invoice) AS Frequency,

SUM(TotalPrice) AS Monetary

FROM

silver_online_retail

GROUP BY

CustomerID;

-- Step 3: Create the Gold Customer Segments Table

CREATE OR REPLACE TABLE gold_customer_segments AS

SELECT

```
CustomerID,  
Recency,  
Frequency,  
Monetary,  
NTILE(4) OVER (ORDER BY Recency DESC) AS R_Score,  
NTILE(4) OVER (ORDER BY Frequency ASC) AS F_Score,  
NTILE(4) OVER (ORDER BY Monetary ASC) AS M_Score  
FROM  
gold_customer_rfm;
```

-- Step 4: Final Segmentation Analysis (for display)

```
SELECT  
CustomerID,  
Recency,  
Frequency,  
Monetary,  
CONCAT(R_Score, F_Score, M_Score) AS RFM_Score_Combined,  
CASE  
    WHEN R_Score = 4 AND F_Score = 4 AND M_Score = 4 THEN 'Champions'  
    WHEN R_Score >= 3 AND F_Score >= 3 THEN 'Loyal Customers'  
    WHEN R_Score >= 3 AND F_Score <= 2 THEN 'Potential Loyalists'  
    WHEN R_Score = 4 AND F_Score = 1 THEN 'New Customers'  
    WHEN R_Score = 2 AND (F_Score = 1 OR F_Score = 2) THEN 'Promising'  
    WHEN R_Score = 3 AND (F_Score = 1 OR F_Score = 2) THEN 'Needs Attention'  
    WHEN R_Score <= 2 AND F_Score >= 3 THEN 'At Risk (High Value)'  
    WHEN R_Score <= 2 AND F_Score <= 2 THEN 'At Risk (Low Value)'  
    WHEN R_Score = 1 THEN 'Lost'  
    ELSE 'Other'
```

```
END AS Customer_Segment  
FROM  
    gold_customer_segments  
ORDER BY  
    Monetary DESC;
```

C. Code for Phase 4: Visualization & Modeling (Python)

```
# 1. Install and import necessary libraries
```

```
%pip install scikit-learn matplotlib seaborn
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
# 2. Load the gold RFM table into a pandas DataFrame
```

```
pdf = spark.table("gold_customer_rfm").toPandas()
```

```
# 3. Data Visualization
```

```
pdf['Monetary_log'] = np.log1p(pdf['Monetary'])
```

```
pdf['Frequency_log'] = np.log1p(pdf['Frequency'])
```

```
# (Code for generating histograms and scatter plots as provided previously)
```

```
# ...
```

```
# 4. Define features (X) and target (y)
```

```
features = ['Recency', 'Frequency']
```

```
X = pdf[features]
```

```
target = 'Monetary'
```

```
y = pdf[target]
```

```
# 5. Split data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# 6. Create and train the model
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
print("--- Machine Learning Model Trained Successfully ---")
```

```
print(f"Model formula: Monetary = (Recency * {model.coef_[0]:.2f}) + (Frequency *  
{model.coef_[1]:.2f}) + {model.intercept_:.2f}\n")
```

```
# 7. Model Evaluation Plotting
```

```
y_pred = model.predict(X_test)
```

```
plt.figure(figsize=(8, 8))
```

```
plt.scatter(y_test, y_pred, alpha=0.3)
```

```
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r', linewidth=2)
```

```
plt.title('Model Performance: Actual vs. Predicted Monetary Value')
```

```
plt.xlabel('Actual Monetary Value (Test Set)')
```

```
plt.ylabel('Predicted Monetary Value')
```

```
plt.show()
```